

ENGINEERING AUDIT — VIDEO PIPELINE

Upload & Playback, End to End

What actually happens between a creator tapping the shutter and a viewer's feed playing that clip back — and where this pipeline still falls short of a TikTok-grade production bar.

Client apps/mobile_flutter (Flutter)

Backend backend/firebase_functions (Cloud Functions v2)

Storage Firestore videos + Cloud Storage videos-raw/

§1 THE PIPELINE, STAGE BY STAGE

Eight stages from shutter to screen. Status reflects what's actually wired today, not what field names imply.

● production-ready ○ exists, incomplete ● not implemented

01	02	03	04
●	●	●	○
Capture	Persist locally	Preview	Publish
Live camera or gallery pick	Copied to app storage, draft saved	Playback + title/caption/tags	Progress + cancel/retry, no rate limit
05	06	07	08
●	●	○	●
Transcode	Moderate	Fetch	Play
No real adaptive-bitrate pipeline	Goes public with zero review	Paginated, preloaded, no CDN	Zero-wait preload + diagnostics

§2 UPLOAD & INGEST

Capture through the moment a document lands in Firestore with `status: 'live'`.

● WORKING

- ✓ **Unified capture screen.** One screen handles live recording (front/rear switch, proper permission flow) and gallery selection, with camera lifecycle correctly torn down and rebuilt across app background/foreground.
- ✓ **Stable local persistence.** Recorded/picked files are copied out of the camera plugin's and picker's ephemeral cache paths before anything else touches them, so a draft survives beyond the OS's whim.
- ✓ **Real draft recovery.** Backing out mid-flow saves the file path and form fields; relaunching Upload offers to resume, not just discard.
- ✓ **Genuine transfer control.** Progress, cancel, and retry are wired to the actual `UploadTask`, not a simulated progress bar — cancel aborts the transfer and deletes the orphaned document.
- ✓ **Automatic thumbnails.** A Cloud Function extracts a real frame via `ffmpeg` on upload, after a missing-binary bug that had silently broken this for every prior upload was found and fixed.

● MISSING FOR PRODUCTION

- ✗ **No client-side compression.** Full-resolution camera output is uploaded byte-for-byte; there's no quality-vs-bandwidth trade-off before the transfer even starts.
TikTok re-encodes on-device before upload; this pipeline sends raw footage over whatever network is available.
- ✗ **No content moderation gate.** A document is created with `visibility: 'public'` immediately — no automated NSFW/violence screening, no human review queue, before it's reachable in the feed.
- ✗ **No malware or duplicate-upload scanning.** Any file passing the extension check is accepted as-is.
- ✗ **No per-user rate limit.** A single account can publish an unbounded number of videos back to back — no daily cap, unlike the existing 50/day quota on the AI job pipeline.
- ✗ **Upload doesn't survive process death.** Cancel/retry work within a live app session, but killing the app mid-transfer loses it — there's no background-session resumability (WorkManager / iOS background URL sessions).

§3 FETCH & PLAYBACK

From a feed query returning documents to a frame appearing on screen.

● WORKING

● MISSING FOR PRODUCTION

- ✓ **Predictive preloading.** A dedicated playback controller preloads the videos adjacent to the current scroll position before the viewer reaches them — a "zero-wait" architecture most apps at this maturity skip entirely.
- ✓ **Real telemetry, not vibes.** Every playback attempt logs time-to-first-frame, whether the preload actually hit, source URL type, and network type — the kind of signal a team needs to know if playback is actually fast, not just feels fast.
- ✓ **Poster frames on load.** Now that thumbnail generation works, the feed shows a real frame instead of a blank tile while a clip buffers.
- ✓ **Resilient field reads.** The client tolerates schema drift across upload paths (legacy vs. current field names for owner, thumbnail, tags) rather than hard-failing on missing data.
- ✗ **No adaptive bitrate.** There's nothing to adapt *to* — the field named `hlsUrl` is, today, the raw MP4's direct download URL. The five-deep URL-priority fallback chain in the video model (`hlsLowUrl` → `hlsStandardUrl` → `hlsUrl` → `videoUrl` → `hlsPath`) is itself the evidence: it exists because a real multi-bitrate manifest has never reliably been there to prefer.
- ✗ **No purpose-built video CDN.** Playback is served straight from Cloud Storage's default path, not an edge network built for video delivery (Cloudflare Stream / Mux / CloudFront-style segment caching).
- ✗ **Static, long-lived URLs.** Playback URLs are download tokens written once into Firestore, not short-lived signed URLs — a leaked link grants indefinite access to that file.
- ✗ **No offline or deep prefetch.** Preloading covers immediate feed neighbors only; nothing is cached for offline viewing or further ahead.
- ✗ **Shallow engagement analytics.** Views and likes are counted, but there's no watch-time percentage, drop-off point, or rewatch signal — the data a recommendation system actually needs.

! IF ONLY THREE THINGS GET FIXED NEXT

Ranked by how directly each blocks a real TikTok-grade launch.

1 Stand up real transcoding.

Without
an
adaptive-
bitrate
ladder,
every
other
playback
optimization
(preloading,
CDN,
diagnostics)
is
polishing
delivery
of a
file
that's
the
wrong
size
for
the
network
it's
on.
This
is the
one
gap
that
makes
every
other
fetch/playback
gap
worse.

2 Gate publish behind moderation.

Instant,
unreviewed
public
visibility
is a
trust
and
safety

liability
at
any
scale
beyond
a
closed
pilot
—
this
needs
to
exist
before
upload
volume
grows,
not
after.

3 Add upload rate limiting and background resumability.

Two
different
failure
modes
with
the
same
root
cause:
nothing
constrains
or
protects
an
in-
flight
or
repeated
upload
once
it
leaves
the
preview
screen.

Scope: apps/mobile_flutter video_upload & video_feed features, backend/firebase_functions video processing, and the deployed Firestore/Storage rules, as of this session. Verify against current code before treating specifics as still accurate – this pipeline has moved fast across several rebuilds.